

Poker Bot Report

Srivanth Guntha
Roll No: 250403

1 Overview

This project implements a heads-up Limit Texas Hold'em poker bot from scratch. The bot estimates its winning chances using Monte Carlo simulation and makes decisions using expected value (EV) and pot odds.

2 State Representation

The bot(P1) represents the current game state using:

- Hole cards
- Community cards
- Pot size
- Amount required to call
- Legal actions
- Current betting street (pre-flop, flop, turn or river)

Instead of manually assigning numeric values to card combinations, the bot computes a single feature: estimated equity.

$$\text{Equity} = P(\text{win}) + \frac{1}{2}P(\text{tie})$$

This value lies between 0 and 1 and summarizes how strong the current hand is against a random opponent.

3 Monte Carlo Equity Estimation

For every decision, the bot repeatedly samples the unknown cards from the remaining deck.

Each simulation:

1. Deals random opponent hole cards.
2. Completes any remaining community cards.
3. Evaluates both hands.
4. Records win, tie or loss.

If there are N simulations,

$$\hat{p} = \frac{\text{Wins} + 0.5 \times \text{Ties}}{N}.$$

This estimated equity is used throughout the game instead of lookup tables.

4 Decision Heuristic

The bot compares its equity with the break-even pot odds.

$$\text{Pot Odds} = \frac{\text{Amount to Call}}{\text{Pot Size} + \text{Amount to Call}}$$

If the estimated equity is above this threshold, calling has positive expected value and the bot calls.

Strong hands above fixed street-specific thresholds are raised for value. Medium-strength hands occasionally bluff with a small probability so the bot does not become too predictable. Weak hands fold when their equity is clearly below the break-even point.

5 Hand Evaluation

A custom evaluator ranks poker hands directly from up to seven cards. It checks hand categories in descending order:

Straight Flush → Four of a Kind → Full House → Flush → Straight → Three of a Kind →
Two Pair → One Pair → High Card

Each hand is represented by a tuple containing the category and tie-breakers, allowing normal tuple comparison to determine the winner.

6 Files

- `agent.py` – Main poker bot logic.
- `card.py` – Card and deck classes.
- `equity.py` – Monte Carlo equity estimation.
- `evaluator.py` – Poker hand evaluator.
- `engine.py` – Local poker engine for playing matches.
- `play_vs_bot.py` – Interactive game against the bot.

7 Playing Against the Bot

To play, run in the terminal:

```
py submission\Srivanth_Guntha_250403\play_vs_bot.py
```

You will be prompted for the number of hands and can enter `fold`, `call`, or `raise` during play.

8 Conclusion

The bot combines a custom poker evaluator, Monte Carlo equity estimation and simple EV-based heuristics into a strategy. While not solving the game optimally, it makes mathematically justified decisions without relying on external poker engines or precomputed data.